

# Uart\_Esp32 - Projektdokumentation - Firmware

## Synchronisierter UART-Image-Transfer mit Konsolenfreigabe fuer sicheren Neustart

Version v0.18

|                                |                                                                                                                                |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Projektname                    | Uart_Esp32                                                                                                                     |
| Dokumentenart                  | Projektdokumentation                                                                                                           |
| Thema                          | Firmware fuer ESP32-WROOM-32 / NodeMCU-32S - UART-Konsole, Decoder, Probe-Runner und MTD-Image-Transfer                        |
| Erstellungsdatum               | 2026-09-21                                                                                                                     |
| Autor / verantwortliche Person | OpenAI ChatGPT (technische Bearbeitung)                                                                                        |
| Version                        | v0.18                                                                                                                          |
| Status                         | Entwurf - lokale C++17-/JavaScript-Pruefungen und Host-Funktionstest erfolgreich; PlatformIO-Build und Hardwaretest ausstehend |
| Aenderungsdatum                | 2026-09-21                                                                                                                     |
| Ablageort                      | Projektordner Uart_ESP32/docs                                                                                                  |
| Referenzen                     | BX3_ESP32_Login_Leitfaden.pdf; Regeln_Projektdokumentation_PDF_DOCX_Pflicht_.pdf; Projektstand v0.17                           |

## Aenderungshistorie

| Version | Datum      | Bearbeiter     | Status  | Aenderung                                                                                     |
|---------|------------|----------------|---------|-----------------------------------------------------------------------------------------------|
| v0.15   | 2026-09-20 | OpenAI ChatGPT | Entwurf | Gemeinsamer UART-Basisbetrieb und getrennte UART-Menues.                                      |
| v0.16   | 2026-09-20 | OpenAI ChatGPT | Entwurf | Autoscroll-Haekchenfeld in der UART-Konsole.                                                  |
| v0.17   | 2026-09-21 | OpenAI ChatGPT | Entwurf | Kopierfreundliche Konsole und SHA-256-gepruefter Image-Transfer.                              |
| v0.18   | 2026-09-21 | OpenAI ChatGPT | Entwurf | Blocksynchronisierung korrigiert und Image-Neustart an bestaetigten Konsolenzugriff gebunden. |

### 1. Ziel der Version v0.18

Version v0.18 korrigiert den in v0.17 beobachteten Zustand **Browserfehler: Block nicht bereit** und ergaenzte einen kontrollierten Neustart nach einer Transferunterbrechung.

Die Kernziele sind:

1. Der Browser darf einen Imageblock erst abrufen, wenn die Firmware genau diesen Block als validiert und browserbereit meldet.
2. Ein noch nicht fertiger Block ist kein Fehler, sondern ein normaler Wartezustand.
3. Ein verlorenes HTTP-ACK darf keinen Block doppelt in die Browserdatei einfuegen und keinen weiteren UART-Block ueberspringen.
4. Nach Abbruch oder nicht wiederherstellbarem Fehler wird die UART-Konsole wieder freigegeben.
5. Ein Neustart des Image-Transfers ist erst moeglich, nachdem der Benutzer den BX3-Shell-Zugriff ueber die UART-Konsole wiederhergestellt und dies dort explizit bestaetigt hat.
6. Der Neustart beginnt fuer ein neues Vollimage bewusst wieder bei Block 0.

WLAN/NTP, MQTT, Batterie/ADC, Deep Sleep, OTA, UART Einstellungen, UART Konsole, UART Decoder und UART Probe-Runner bleiben erhalten.

### 2. Ursache der Meldung "Block nicht bereit"

In v0.17 verwendete die Browserseite im Wesentlichen eine einzelne aktuelle Blocknummer. Zwischen der Statusabfrage und dem eigentlichen HTTP-Abruf konnte der ESP32 jedoch noch im Empfangs- oder Pruefzustand sein. Der Browser versuchte dadurch gelegentlich /uart/image/block abzurufen, obwohl der Block noch nicht im Zustand BlockReady war.

Die bisherige Route antwortete dann mit HTTP 409 und dem Text:

```
Block nicht bereit
```

Auf der Weboberflaeche erschien dies als Browserfehler, obwohl der UART-Transfer anschliessend normal weiterlaufen konnte.

### 3. Getrennte Blockzustaende

v0.18 trennt jetzt drei Werte:

| Wert          | Bedeutung                                                                      |
|---------------|--------------------------------------------------------------------------------|
| current_block | Block, den die Image-Logik aktuell verarbeitet bzw. als naechstes verarbeitet. |

| Wert             | Bedeutung                                                                                     |
|------------------|-----------------------------------------------------------------------------------------------|
| ready_block      | Vollstaendig empfangener und SHA-256-validierter Block, der vom Browser abgeholt werden darf. |
| last_acked_block | Letzter Block, den der Browser erfolgreich bestaetigt hat.                                    |

Wenn kein browserbereiter oder bestaetigter Block existiert, liefert die Status-API fuer den jeweiligen Wert -1.

Die Weboberflaeche zeigt diese Werte getrennt als **Empfangsblock**, **Browserbereit** und **Letztes ACK** an.

## 4. Blockabruf als normaler Wartezustand

Die Route

```
GET /uart/image/block?block=N
```

liefert den Binärblock nur, wenn N exakt dem aktuell validierten ready\_block entspricht.

Ist der Block noch nicht fertig oder stimmt die angeforderte Blocknummer nicht, antwortet der ESP32 nicht mehr mit einem sichtbaren Fehler, sondern mit:

```
HTTP 202 Accepted
```

und einem kleinen JSON-Wartezustand.

Der Browser interpretiert HTTP 202 als **noch nicht bereit** und wartet auf die naechste Statusabfrage. Damit ist ein kurzfristiger Timingunterschied zwischen Browser und ESP32 kein Fehler mehr.

## 5. Robustes ACK-Verfahren

Der Browser uebernimmt einen validierten Block in zwei Stufen:

```
GET /uart/image/block?block=N
↓
Block liegt als Uint8Array im Browser vor
↓
POST /uart/image/ack block=N
↓
ACK erfolgreich
↓
Block wird genau einmal in die Browser-Chunkliste aufgenommen
```

Die Firmware bindet das ACK an die konkrete Blocknummer.

Zusaetzlich ist das ACK idempotent: Wenn der ESP32 Block N bereits bestaetigt hat, die HTTP-Antwort aber beim Browser verloren ging, darf der Browser dasselbe ACK fuer N erneut senden. Die Firmware bestaetigt es erneut, ohne einen weiteren Block zu ueberspringen.

Der Browser behaelt einen noch nicht bestaetigten Block in pendingBytes und versucht nur das ACK erneut. Er laedt denselben Block dadurch nicht mehrfach in die spaetere Image-Datei.

## 6. Fehleraufschluesselung

Neben den bisherigen Gesamtzaehlern werden Fehler jetzt nach Ursache aufgeschluesselt:

- UART Timeout
- SHA-256
- Base64
- Marker / Synchronisierung
- Groesse / Laenge / Puffer
- Sonstige

Die Weboberflaeche zeigt diese Zaehler direkt im Image-Status an. Damit ist bei einem laengeren Transfer besser erkennbar, ob beispielsweise Markerprobleme oder echte Integritaetsfehler auftreten.

Die bestehende Grenze von maximal 5 Wiederholungen pro Block bleibt erhalten.

## 7. Verhalten bei Abbruch oder nicht wiederherstellbarem Fehler

Bei einem manuellen Abbruch oder einem Fehler, der nicht mehr automatisch behandelt werden kann, fuehrt die Firmware weiterhin die temporaere Bereinigung aus:

```
rm -f /tmp/bx3blk
```

Danach wird der Image-Zustand auf Fehler gesetzt und die exklusive UART-Belegung des Image-Transfers endet. Dadurch gelten wieder die normalen UART-Regeln:

```
UART RX -> Konsole / Rohdaten / Decoder
UART TX -> wieder fuer Konsole verfuegbar, sofern unter UART Einstellungen freigegeben
Probe-Runner -> wieder moeglich
```

Der Statustext weist ausdruecklich darauf hin, dass vor einem Image-Neustart zuerst die Konsole verwendet werden muss.

## 8. Konsolengesteuerte Neustartfreigabe

Nach einer Unterbrechung setzt die Firmware intern:

```
restart_required = true
console_confirmed = false
```

Die normale Startfunktion des Image-Transfers lehnt jetzt einen erneuten Start ab, solange `console_confirmed` nicht gesetzt wurde.

Auf der Seite **UART Konsole** erscheint nur in diesem Zustand ein zusaetzlicher Bereich **Image-Transfer nach Unterbrechung**.

Der vorgesehene Ablauf ist:

1. UART Image-Transfer wurde unterbrochen.
2. Die Firmware gibt die UART-Konsole wieder frei.
3. Benutzer oeffnet **UART Konsole**.
4. Benutzer stellt den BX3-Login bzw. Shell-Zugriff wieder her.
5. Benutzer drueckt **Konsolenzugriff hergestellt - Image-Neustart freigeben**.
6. Die Firmware setzt `console_confirmed = true`.
7. Auf der Seite **UART Image-Transfer** wird **Transfer komplett neu starten (Block 0)** freigeschaltet.

**Wichtig:** Die Firmware kann nicht verlaesslich erkennen, ob der angezeigte Shell-Prompt tatsaechlich ein erfolgreicher Login ist. Die Schaltflaeche ist deshalb eine bewusste Benutzerbestaetigung nach manuell geprueftem Konsolenzugriff.

## 9. Warum der Neustart bei Block 0 beginnt

v0.17/v0.18 sammelt die validierten 32-KiB-Blocke fuer das fertige Image im Browser-RAM. Fuer die geforderte Wiederherstellung des Konsolenzugriffs muss der Benutzer die UART-Konsole oeffnen; dabei kann die Image-Seite verlassen werden und ihr Browserpuffer verloren gehen.

Darum startet die neue explizite Neustartfunktion absichtlich bei:

```
Block 0
```

Damit entsteht erneut eine zusammenhaengende Vollimage-Datei, deren Gesamt-SHA256 am Ende mit dem BX3 verglichen werden kann.

Das vorhandene manuelle Feld **Startblock** bleibt fuer bewusste Teilimages erhalten. Es ist jedoch kein automatisches Zusammenfuegen einer vorher teilweise geladenen Browserdatei.

## 10. Neue und geaenderte HTTP-Routen

| Route              | Methode | Zweck                                                                                         |
|--------------------|---------|-----------------------------------------------------------------------------------------------|
| /uart/image/status | GET     | Liefert Phase, Empfangsblock, ready_block, last_acked_block, Fehlerzaehler und Restartstatus. |

| Route                       | Methode | Zweck                                                                                     |
|-----------------------------|---------|-------------------------------------------------------------------------------------------|
| /uart/image/block?block=N   | GET     | Liefert nur exakt den validierten Block N; sonst HTTP 202.                                |
| /uart/image/ack             | POST    | Blocknummerngebundenes und idempotentes ACK.                                              |
| /uart/image/abort           | POST    | Bricht Transfer ab und fordert danach Konsolenfreigabe vor Neustart.                      |
| /uart/image/console-confirm | POST    | Explizite Bestaetigung, dass der Shell-Zugriff ueber die Konsole wiederhergestellt wurde. |
| /uart/image/restart         | POST    | Startet nach erfolgreicher Konsolenbestaetigung ein neues Vollimage ab Block 0.           |

## 11. Status-API v0.18

Die Image-Status-API enthaelt unter anderem:

```

phase
block_ready
current_block
ready_block
last_acked_block
restart_required
console_confirmed
restart_allowed
error_timeout
error_sha
error_base64
error_marker
error_size
error_other

```

Damit kann die Browseroberflaeche ihren Zustand aus der Firmware ableiten, statt aus einer einzigen Blocknummer zu schliessen, ob ein Download bereits moeglich ist.

## 12. Unveraenderte Sicherheitslogik

Die grundlegende Image-Sicherheitslogik aus v0.17 bleibt erhalten:

- nur Quellen nach /dev/mtdNro mit N = 0...31,
- read-only Zielpfad als Quelle,
- 32-KiB-Blocke,
- Groessenpruefung pro Block,
- SHA-256 pro Block,
- Gesamt-SHA256 bei Volltransfer ab Block 0,
- maximal 5 Wiederholungen pro Block,
- Image-Transfer reserviert UART TX/RX waehrend er aktiv ist,
- Konsole, Probe-Runner und Feld-5-Replay koennen waehrend eines aktiven Transfers nicht dazwischen senden.

Die Hardwarehinweise aus dem BX3-Konsolenleitfaden bleiben ebenfalls gueltig: gemeinsame Masse, Pegelpruefung, kein VCC-Verbund und TX bei der Erprobung nur ueber geeignete Schutzmassnahmen.

## 13. Geaenderte Dateien

| Datei             | Aenderung                                                              |
|-------------------|------------------------------------------------------------------------|
| src/UartManager.h | Neue Ready-/ACK-/Restart-Zustaende, Restartfreigabe und Fehlerzaehler. |

| Datei                                                                                         | Aenderung                                                                                                       |
|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| src/UartManager.cpp                                                                           | Blocknummerngebundenes idempotentes ACK, Fehlerklassifikation, Konsolenbestaetigung und Neustartsperr.          |
| src/main.cpp                                                                                  | v0.18-Weboberflaeche, HTTP-202-Wartezustand, robuste Browser-ACK-Logik, Konsolenbestaetigung und Restart-Route. |
| src/ConfigDefaults.h                                                                          | Firmwareversion auf 0.18.0 angehoben.                                                                           |
| README.md                                                                                     | v0.18-Aenderungen beschrieben.                                                                                  |
| docs/2026-09-21 - Uart_Esp32 - Projektdokumentation - Firmware - v0.18.md                     | Neue Projektdokumentation.                                                                                      |
| docs/2026-09-21 - Uart_Esp32 - Projektdokumentation - Firmware - v0.18.docx                   | Bearbeitbare Dokumentation.                                                                                     |
| docs/2026-09-21 - Uart_Esp32 - Projektdokumentation - Firmware - v0.18.pdf                    | Vorrangige PDF-Dokumentation.                                                                                   |
| docs/2026-09-21 - Uart_Esp32 - Commit-Nachricht - Image-Transfer-Synchronisierung - v0.18.txt | Verpflichtende englische Commit-Nachricht.                                                                      |
| VALIDATION_v0.18.txt                                                                          | Validierungsstand der Auslieferung.                                                                             |

## 14. Validierung

Durchgefuehrt wurden:

- lokale g++ -std=c++17 -fsyntax-only-Pruefung fuer UartManager.cpp, TestCandidateCatalog.cpp und main.cpp mit den vorhandenen ESP32-/Webserver-Host-Stubs,
- Extraktion aller eingebetteten JavaScript-Blöcke aus main.cpp und Pruefung mit node --check,
- Host-Funktionstest fuer den Neustartablauf: Transferstart -> Unterbrechung -> Neustart gesperrt -> Konsolenzugriff bestaetigt -> Neustart erlaubt,
- ZIP-Struktur- und Integritaetspruefung der Auslieferung,
- visuelle Renderpruefung von DOCX und PDF.

Noch ausstehend:

- echter PlatformIO-Build,
- Flash-Test auf dem Ziel-ESP32,
- realer BX3-End-to-End-Test,
- provozierter WLAN-/HTTP-Unterbrechungstest waehrend eines 16-MiB-Transfers,
- unabhengiger SHA-256-Vergleich der fertigen Image-Datei auf dem PC.

## 15. Testcheckliste am BX3

- [ ] pio run -e uart\_esp32\_usb erfolgreich.
- [ ] Firmware v0.18 auf Ziel-ESP32 geflasht.
- [ ] UART-Konsole mit BX3 funktioniert bidirektional.
- [ ] Image-Transfer startet auf /dev/mtd7ro.
- [ ] Empfangsblock und browserbereiter Block werden getrennt angezeigt.
- [ ] Kein sichtbarer Fehler mehr, nur weil ein Block noch nicht bereit ist.
- [ ] Ein absichtlich abgebrochener Transfer gibt die UART-Konsole frei.
- [ ] Image-Neustart ist vor Konsolenbestaetigung gesperrt.
- [ ] BX3-Shell-Zugriff kann ueber die Konsole wiederhergestellt werden.
- [ ] Bestaetigung in der Konsole schaltet den Image-Neustart frei.
- [ ] Neustart beginnt bei Block 0.
- [ ] Volltransfer endet mit IMAGE VERIFIED.

[ ] SHA-256 der heruntergeladenen Datei stimmt mit sha256sum /dev/mtd7ro ueberein.

## 16. Commit-Nachricht

BL\_Fix image transfer synchronization and restart gating

Date: 2026-09-21

Summary:

- Separated receiving, browser-ready and acknowledged image block state.
- Changed premature block requests into HTTP 202 wait responses.
- Added block-bound idempotent acknowledgements to prevent duplicate browser chunks.
- Added categorized image-transfer error counters.
- Released the UART console after interrupted transfers and required explicit console-access confirmation before an image restart.
- Added a clean full-image restart from block 0 after console confirmation.
- Kept existing WLAN, MQTT, OTA, battery, deep-sleep, decoder and probe-runner functions unchanged.

Changed files:

- src/UartManager.h
- src/UartManager.cpp
- src/main.cpp
- src/ConfigDefaults.h
- README.md
- docs/2026-09-21 - Uart\_Esp32 - Projektdokumentation - Firmware - v0.18.md
- docs/2026-09-21 - Uart\_Esp32 - Projektdokumentation - Firmware - v0.18.docx
- docs/2026-09-21 - Uart\_Esp32 - Projektdokumentation - Firmware - v0.18.pdf
- docs/2026-09-21 - Uart\_Esp32 - Commit-Nachricht - Image-Transfer-Synchronisierung - v0.18.txt
- VALIDATION\_v0.18.txt